STREAMLIT PRACTICAL EXAMINATION

Interactive Dashboard and Data Exploration

Maximum Marks: 50 **Duration: 1 hr**

Dataset: Top 250 Movies on IMDb in 2026
Dataset Link: Download IMDb Top 250 Movies Dataset

General Instructions

1. Download the given dataset from the provided Kaggle link.
2. Develop the application using Python and Streamlit.
3. Use Pandas for loading, filtering, sorting, and analyzing the dataset.
4. All visualizations must be generated using appropriate charting functions.
5. Use appropriate titles, labels, and layouts throughout the application.
6. The dashboard should update dynamically whenever a filter or widget value changes.
7. Display meaningful information and success messages wherever appropriate.
8. Ensure that the application executes without errors.

1. Section A – Dataset Loading and Preparation

5 Marks

Question 1. Dataset Loading and Initial Exploration

Create a Streamlit application and load the IMDb Top 250 Movies dataset.
Perform the following tasks:

a) Load the CSV dataset into a Pandas DataFrame and display the first 10 records using an appropriate Streamlit component. (1)

b) Display the following dataset information:

- Total number of rows and columns.
- Column names.
- Data types of columns.

(2)

c) Check for missing values and display the number of missing values in each column. (1)

d) Display an appropriate success message after successfully loading the dataset. (1)

Hints

- Use `pd.read_csv()` to load the dataset.
- Use `df.head()` to display the first 10 records.
- Use `df.shape`, `df.columns`, and `df.dtypes`.
- Use `df.isnull().sum()` to identify missing values.
- Explore `st.dataframe()`, `st.write()`, and `st.success()`.

2. Section B – Movie Analytics Dashboard

18 Marks

Question 2. Dashboard KPI Metrics

7 Marks

Create a Movie Analytics Dashboard displaying the following Key Performance Indicators (KPIs). Display the following metrics:

a) Total number of movies available in the dataset. (1)

b) Average IMDb rating of all movies using the `averageRating` column. (2)

c) Highest IMDb rating available in the dataset. (1)

d) Total number of votes across all movies using the `numVotes` column. (2)

e) Display all four KPI metrics in a single row using Streamlit columns. (1)

Hints

- Use Pandas aggregation functions:
  - `len(df)` or `df.shape[0]`
  - `df['averageRating'].mean()`
  - `df['averageRating'].max()`
  - `df['numVotes'].sum()`
- Use `st.columns(4)` to create four columns.
- Use `st.metric()` to display each KPI.

Question 3. Data Visualization

8 Marks

Create the following visualizations using the IMDb dataset.

a) Create a bar chart showing the number of movies belonging to different genres. (2)

b) Create a histogram showing the distribution of IMDb ratings using the `averageRating` column. (2)

c) Create a scatter plot showing the relationship between `averageRating` and `numVotes`. (2)

d) Create a bar chart showing the top 10 movies based on `grossWorldwide`. Display movie titles and their corresponding worldwide gross collections. (2)

Hints

- For genre analysis, use `explode()` if multiple genres are stored in a single cell, followed by `value_counts()`.
- Use `st.bar_chart()` or an appropriate charting library for bar charts.
- Use `st.hist()` or `st.bar_chart()` with appropriate bins for the rating distribution.
- Use a scatter plot with:
  - X-axis: `averageRating`
  - Y-axis: `numVotes`
- For worldwide gross collection:
  - Sort the dataset using `sort_values()`.
  - Use `ascending=False`.
  - Select the first 10 records using `head(10)`.
- Add appropriate chart titles and axis labels.

Question 4. Dashboard Layout and Additional Information

3 Marks

Improve the dashboard layout using the following Streamlit components:

a) Use `st.tabs()` to organize the visualizations into separate tabs. (1)

b) Use `st.expander()` to display additional information about the dataset, such as the number of records, columns, and available movie information. (1)

c) Use `st.divider()` to separate dashboard sections and display appropriate information or success messages. (1)

Hints

- Create separate tabs for genre analysis, rating analysis, and gross collection analysis.
- Use `with st.expander("Dataset Information"):` to display additional information.
- Use `st.divider()` between KPI metrics and visualizations.
- Use `st.info()` and `st.success()` for messages.

3. Section C – Interactive Data Exploration and Filtering

27 Marks

Question 5. Interactive Movie Explorer

12 Marks

Create an interactive Movie Explorer page that allows users to search, filter, and sort movies using Streamlit widgets.

Implement the following filters:

a) **Selectbox:** Allow users to select a movie genre and display movies belonging to the selected genre. (2)

b) **Multiselect:** Allow users to select multiple countries of origin and filter movies based on the selected countries. (2)

c) **Slider:** Provide a rating slider to filter movies based on their IMDb ratings. (2)

d) **Number Input:** Allow users to specify the minimum number of votes required for displaying movies. (1)

e) **Text Input:** Provide a search box to search for movies using their titles. (1)

f) **Radio Button:** Allow users to select the sorting order of IMDb ratings:

- Ascending
- Descending

(1)

g) Display the total number of movies remaining after applying all selected filters. (1)

h) Display the final filtered DataFrame using `st.dataframe()` and provide a reset button to clear the filters. (2)

Hints

Suggested implementation approach:

- Create a copy of the original DataFrame before applying filters:

```
filtered_df = df.copy()
```

- Use `st.selectbox()` to select a genre.
- Use `st.multiselect()` to select countries.
- Use `st.slider()` to select the minimum and maximum IMDb ratings.
- Use `st.number_input()` to specify minimum votes.
- Use `st.text_input()` to search movie titles.
- Apply conditions to the filtered DataFrame:

```
filtered_df = filtered_df[
    filtered_df['averageRating'] >= min_rating
]
```

- For movie title searching, explore:

```
str.contains()
```

- For sorting, explore:

```
sort_values()
```

- Use `st.radio()` to select ascending or descending order.
- Use `st.button()` to create a reset button.
- Display the number of filtered records using `len(filtered_df)`.

Important: Apply all selected filters to the same filtered DataFrame so that the results reflect every selected condition.

Question 6. Session State and Form Handling

10 Marks

Implement session state and form handling to create an interactive movie search experience. Perform the following tasks:

a) Use `st.session_state` to maintain a selected movie or user preference across Streamlit interactions. (2)

b) Create a form using `st.form()` to collect movie search preferences. The form should contain:

- Movie title.
- Preferred genre.
- Minimum IMDb rating.

(3)

c) Add a submit button inside the form using `st.form_submit_button()`. Process the entered preferences only after form submission. (2)

d) Display the selected movie information after successful form submission. Display relevant information such as:

- Movie title.
- Genre.
- IMDb rating.
- Number of votes.

(2)

e) Display an appropriate success or information message based on the search result. (1)

Hints

Suggested implementation approach:

- Initialize a session state variable before using it:

```
if 'selected_movie' not in st.session_state:
    st.session_state.selected_movie = None
```

- Create a form:

```
with st.form("movie_search_form"):
    # Add input widgets here

    submitted = st.form_submit_button(
        "Search Movie"
    )
```

- Use `st.text_input()`, `st.selectbox()`, and `st.slider()` inside the form.
- Check whether the form has been submitted:

```
if submitted:
    # Process search preferences
```

- Use Pandas filtering conditions to find matching movies.
- Store the selected movie information in `st.session_state`.
- Display the stored information using `st.write()`, `st.dataframe()`, or `st.metric()`.
- Use `st.success()` when a matching movie is found and `st.info()` when no matching movie is available.

Additional Implementation Guidance

Suggested Application Structure
The application can be organized into the following sections:

1. Dataset Loading and Preparation
2. Movie Analytics Dashboard
3. KPI Metrics
4. Data Visualization
5. Interactive Movie Explorer
6. Session State and Movie Search Form

Important Streamlit Functions

The following functions may be useful while developing the application.

Function	Purpose
`st.title()`	Display the application title.
`st.header()`	Display section headings.
`st.subheader()`	Display subsection headings.
`st.columns()`	Organize content into columns.
`st.metric()`	Display KPI metrics.
`st.tabs()`	Organize content into tabs.
`st.expander()`	Show or hide additional information.
`st.divider()`	Separate different sections.
`st.progress()`	Display a progress indicator.
`st.info()`	Display information messages.
`st.success()`	Display success messages.
`st.dataframe()`	Display tabular data.
`st.selectbox()`	Select a single option.
`st.multiselect()`	Select multiple options.
`st.slider()`	Select a numerical range.
`st.number_input()`	Enter numerical values.
`st.text_input()`	Enter text.
`st.radio()`	Select one option from multiple choices.
`st.button()`	Create an interactive button.
`st.session_state`	Maintain information across reruns.
`st.form()`	Group input widgets into a form.

Submission Requirements

1. Submit the complete Python source code (`.py` file).
2. Submit the screenshots of the developed dashboard and its visualizations.
3. Ensure that all filters, buttons, tabs, and form components are working correctly.
4. The application must run successfully using Streamlit.

End of Question Paper

Maximum Marks: 50